

P2115R0: US069: Merging of multiple definitions for unnamed unscoped enumerations

This paper resolves NB Comment US069:

Unnamed unscoped enumerations may be defined in multiple header units, and have no linkage. The merging rules of multiple definitions from header units or appearing textually outside of module purview require implementations to determine if two particular definitions are for the same entity. There is no mechanism specified to determine whether two such enumerations are for the same entity. Unnamed, untyped, enums are common in header files, as the enumeration values also appear in the containing (namespace) scope. A mechanism should be specified.

Proposed change:

Use the first enumerator as the key. If two unnamed unscoped enumeration definitions in the same scope have the same identifier for their first enumerator, they are defining the same enumerated type. (It therefore is an ODR violation if the enumerators are not the same.) FYI this is the heuristic independently implemented in the Clang Modules extension and GCC C++ Modules. It is expected in Clang C++ Modules.

Discussion:

This is an existing ODR problem, made more pressing by modules. EWG attempted guidance:

Use the names of the first enumerator to merge enum types across translation units, as recommended by US069

SF:1 F:5 N:4 A:2 SA:3. Not consensus

Treat the enumerators of anonymous enums as the underlying type, as in C

SF:0 F:1 N:2 A:5 SA:8 Not consensus

EWG was concerned that the second option would not permit using an enumerator of an anonymous enum as a deduced (or decltype) template type parameter:

```
template<auto V> int Frob () {return int (V);}
enum { A = 195'936'478 };
template int Frob<A> ();
```

This is already an ODR violation, and GCC, Clang and MSVC++ all produce different, clearly broken, manglings for the instantiation. In GCC's case it is '_Z4FrobIL8._anon_0195936478EEiv', exposing the internal anonymous identifier. Clang similarly exposes an internal counter-generated name. MSVC++, I believe, attempts to produce a globally unique identifier. If any existing code is doing this, it is very brittle!

EWG does request the creation of a CWG issue and we recommend that issue be resolved as a DR.

CWG requested drafting for the mechanism proposed in US069 (the first of the above two options).

This suggests that for:

```
namespace X { enum { A }; }
```

the enum type would be mangled as `_ZN1X1AE` (to pick an ABI at random). Fortunately that ABI never needs to mangle the enumerator itself -- when encoded as template value parameters a type/integral value pair is used.

Core discussed this direction in Prague, and was concerned that two different (non-header) translation units may define anonymous enums (for instance ``enum { INIT, ... };``) that now become an ODR violation, whereas before they did not — they had internal linkage. This would also apply to an anonymous enum that had enumerators added in a version2 header. Thus changes to the ODR rules were also requested.

Wording:

Alter [basic.def.odr] (6.3) para 12:

In the first part of the paragraph ...

There can be more than one definition of a

- class type (Clause 11),
- enumeration type (9.7.1),
- inline function or variable (9.2.7),
- templated entity (13.1),
- default argument for a parameter (for a function in a given scope) (9.3.3.6), or
- default template argument (13.2)

in a program provided that each definition appears in a different translation unit, and ~~provided~~ the definitions satisfy the following requirements. ~~There shall not be more than one definition of an entity that is attached to a named module (10.1); no diagnostic is required unless a prior definition is reachable at a point where a later definition appears.~~ Given such an entity ~~named~~ D defined in more than one translation unit, for all definitions of D, or, if D is unnamed enumeration, for all definitions of D that are reachable at each program point, ~~all of~~ the following requirements shall be satisfied.

- Each such definition of D shall not be attached to a named module (10.1).
- Each such definition of D shall consist of the same sequence of tokens, where the definition of a closure type is considered to consist of the sequence of tokens of the corresponding *lambda-expression*.
- ...

Editing instruction: alter each item in the bulleted list as:

- Each such definition of D

Split the block of the paragraph following bullet 12.14, (creating a new paragraph):

If D is a template and is defined in more than one translation unit, then the preceding requirements shall apply both to names from the template's enclosing scope used in the template definition (13.8.3), and also to dependent names at the point of instantiation (13.8.2). These requirements also apply to corresponding entities defined within each definition of D (including the closure types of *lambda-expressions*, but excluding entities defined within default arguments or default template arguments of either D or an entity not defined within D). For each such entity and for D itself, the behavior is as if there is a single entity with a single definition, including in the application of these requirements to other entities. [Note: The entity is still declared in multiple translation units, and 6.6 still applies to these declarations. In particular, *lambda-expressions* (7.5.5) appearing in the type of D may result in the different declarations having distinct types, and *lambda-expressions* appearing in a default argument of D may still denote different types in different translation units. — end note]

If ~~these~~ definitions of ~~D~~ do not satisfy these requirements, then the program is ill-formed; a diagnostic is required only if the entity is attached to a named module and a prior definition is reachable at the point where a later definition occurs. [Example: ...

Insert a new paragraph after the example at the end of [basic.def.odr] para 12:

If, at any point in the program, there is more than one reachable unnamed enumeration definition in the same scope that have the same first enumerator name and do not have typedef names for linkage purposes (9.7.1), those unnamed enumeration types shall be the same; no diagnostic required

Add to the bullet list in [basic.link] (6.6) para 5:

An unnamed namespace or a namespace declared directly or indirectly within an unnamed namespace has internal linkage. All other namespaces have external linkage. A name having namespace scope that has not been given internal linkage above and that is the name of

- a variable; or
- a function; or
- a named class (11.1), or an unnamed class defined in a typedef declaration in which the class has the typedef name for linkage purposes (9.2.3); or
- a named enumeration (9.7.1), or an unnamed enumeration defined in a typedef declaration in which the enumeration has the typedef name for linkage purposes (9.2.3); or
- an unnamed enumeration that has an enumerator as a name for linkage purposes (9.7.1); or
- a template

Alter [basic.link] (6.6) para 10 after the list:

If multiple declarations of the same name with external linkage would declare the same entity except that they are attached to different modules, the program is ill-formed; no diagnostic is required.

[Note: *using-declarations*, typedef declarations, and *alias-declarations* do not declare entities, but merely introduce synonyms. Similarly, *using-directives* do not declare entities. Enumerators do not have linkage, but may serve as the name of an enumeration with linkage (9.7.1) — end note]

Change [dcl.typedef] 9.2.3 para 9

If the typedef declaration defines an unnamed class ~~(or enum)~~ or enumeration, the first typedef-name declared by the declaration to be that ~~class type (or enum type)~~ is used to denote the ~~class type (or enum type)~~ for linkage purposes only (6.6).

Alter [dcl.enum] (9.7.1) para 11:

Each *enum-name* and each unscoped *enumerator* is declared in the scope that immediately contains the *enum-specifier*. Each scoped enumerator is declared in the scope of the enumeration. An unnamed enumeration that does not have a typedef name for linkage purposes (9.2.3) and that has a first enumerator is denoted, for linkage purposes (6.6), by its underlying type and its first enumerator; such an enumeration is said to have an enumerator as a name for linkage purposes. These names obey the scope rules defined for all names in 6.4 and 6.5. [Note: Each unnamed enumeration with no enumerators is a distinct type. — end note]